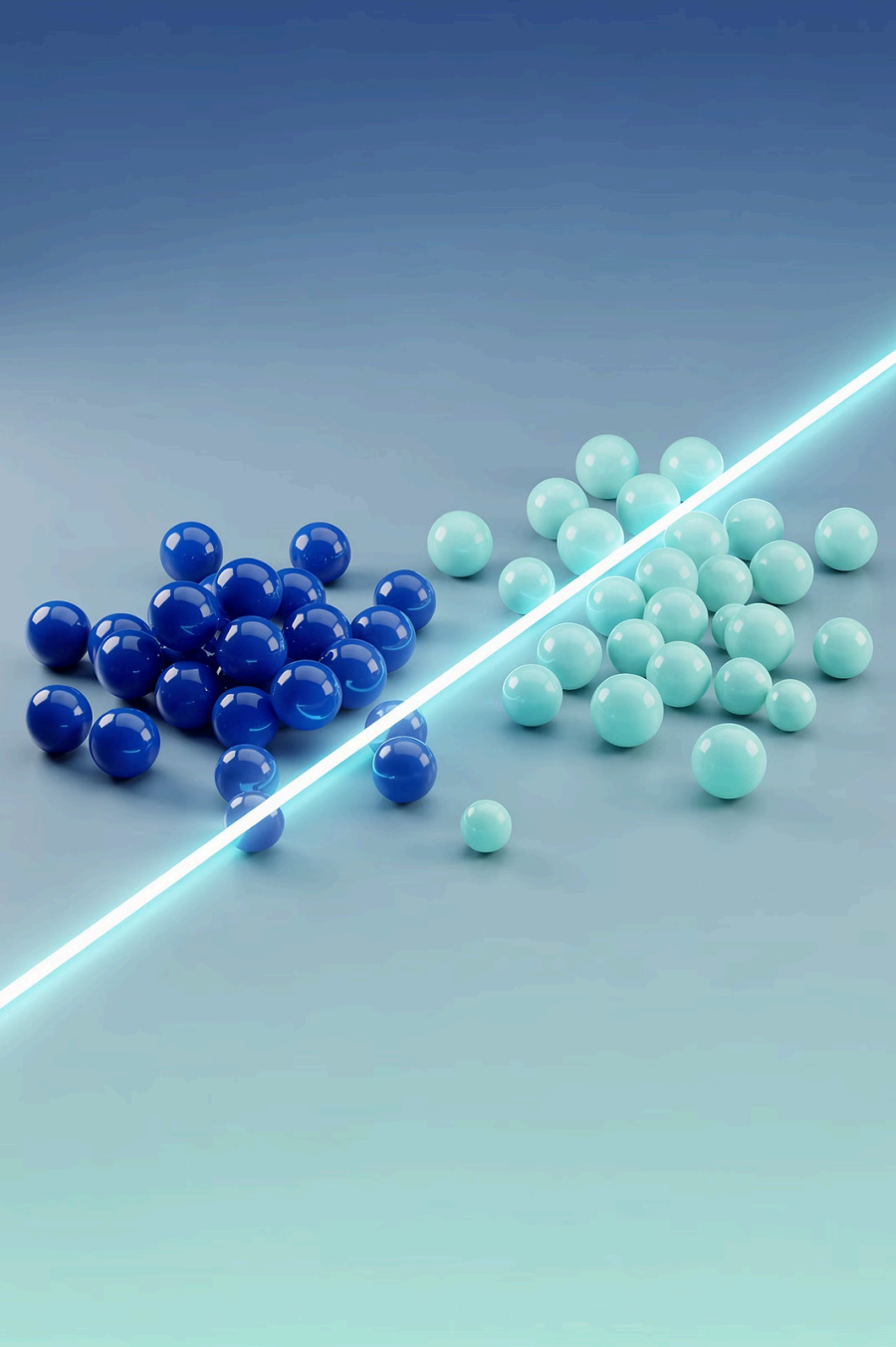




El Perceptrón Simple: El Nacimiento del Aprendizaje Automático

En 1958, Frank Rosenblatt introdujo el Perceptrón, transformando la neurona artificial en un algoritmo de aprendizaje supervisado. Este modelo sentó las bases de la **Inteligencia Artificial moderna** y la Ciencia de Datos, siendo el clasificador lineal más básico capaz de aprender por sí mismo mediante ensayo y error.



¿Qué es el Perceptrón?

Clasificador Lineal

Toma un conjunto de datos de entrada y decide a qué categoría pertenecen, trazando una **frontera de decisión** en el espacio de datos.

Aprendizaje vs. Programación Tradicional

A diferencia de un programa clásico donde un humano escribe reglas del tipo **SI... ENTONCES**, el Perceptrón **descubre esas reglas** mediante un proceso iterativo de retroalimentación. El modelo de McCulloch-Pitts realizaba cálculos lógicos, pero carecía de la capacidad de ajustarse por sí mismo basándose en la experiencia.

Arquitectura Interna: Entradas y Pesos Sinápticos

El modelo recibe un vector de entradas $(x_1, x_2, \dots, x_n)$, donde cada valor representa una característica del problema (p. ej., presión arterial o nivel de glucosa en un diagnóstico médico).

Pesos Positivos y Grandes

La característica es muy importante para que el resultado sea **1**. Impulsa la activación de la neurona.

Pesos Negativos

La característica "**resta**" e impulsa el resultado hacia el **0**. Inhibe la activación.

El Aprendizaje

Consiste exclusivamente en encontrar los **valores correctos** para estos pesos sinápticos $(w_1, w_2, \dots, w_n)$.

Función de Agregación y Sesgo (Bias)

El nodo central realiza el **producto escalar**: multiplica cada entrada por su peso y suma todos los resultados. Esta suma representa el nivel de excitación total del sistema.

$$Z = (x_1 \cdot w_1) + (x_2 \cdot w_2) + \dots + (x_n \cdot w_n)$$

El **Bias (b)** es un peso adicional independiente de las entradas. Sin él, la frontera de decisión estaría obligada a pasar por el origen (0,0). El Bias permite desplazar el umbral de activación para adaptarse mejor a la distribución real de los datos. La fórmula completa queda:

$$Z = \sum_{i=1}^n (x_i \cdot w_i) + b$$

El Algoritmo de Aprendizaje: Regla de Actualización



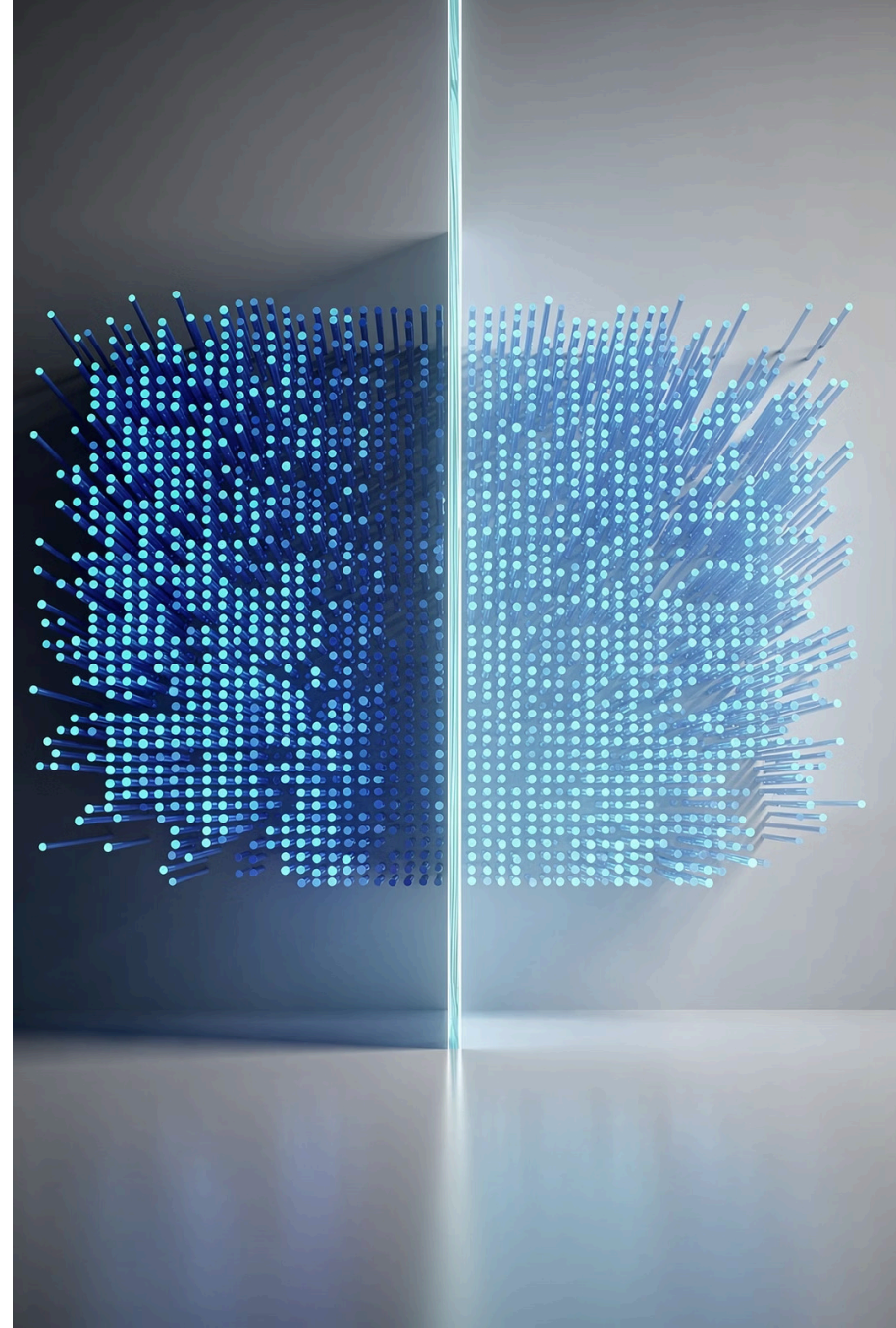
La **Tasa de Aprendizaje** define el tamaño de cada corrección. Si es muy alta, el modelo aprende rápido pero puede ser inestable; si es muy baja, el aprendizaje es lento pero más preciso. La regla de actualización es:

$$\text{Nuevo Peso} = \text{Peso Anterior} + (\eta \cdot \text{Error} \cdot x_i)$$

Geometría del Perceptrón: Separabilidad Lineal

El Perceptrón crea un **Hiperplano de Decisión**: una línea en 2D, un plano en 3D. Su mayor limitación es que solo resuelve problemas **linealmente separables**. Si los datos están mezclados (como en el problema lógico **XOR**), una sola línea nunca podrá dividirlos correctamente.

⚠ Esta limitación provocó el primer "invierno de la IA" en los años 70, hasta que se descubrió que conectar muchos Perceptrones en capas permite trazar fronteras de decisión curvas y complejas.



Importancia en el Modelizado Actual

Aunque hoy utilizamos modelos mucho más complejos, el Perceptrón simple sigue siendo la **unidad básica de aprendizaje**. Entender cómo un simple ajuste de pesos basado en el error puede llevar a una máquina a "entender" un patrón es el fundamento de todas las arquitecturas de **Deep Learning** actuales.

El Perceptrón nos enseñó que la inteligencia no necesita reglas mágicas, sino una estructura matemática capaz de corregirse a sí misma.

Perceptrón Simple

Clasificador lineal, 1 neurona

Redes Multicapa

Fronteras curvas y complejas

Deep Learning

Cientos de capas, millones de pesos

Funciones de Activación: El Mecanismo de Decisión No Lineal

La **función de activación** transforma la suma ponderada de las entradas en una señal de salida utilizable. Sin no linealidad, una red con múltiples capas sería equivalente a una sola neurona simple, ya que la suma de funciones lineales siempre produce otra función lineal. Las funciones de activación **rompen esta linealidad**, convirtiendo la red en un **aproximador universal de funciones**.



Función Escalón y Función Sigmoides

Función Escalón (Heaviside)

Base del Perceptrón original. Esquema estrictamente binario.

$$f(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{si } z < 0 \end{cases}$$

⚠ No es derivable, lo que impide el uso del descenso de gradiente para ajustar pesos de forma suave.

Función Sigmoides (Logística)

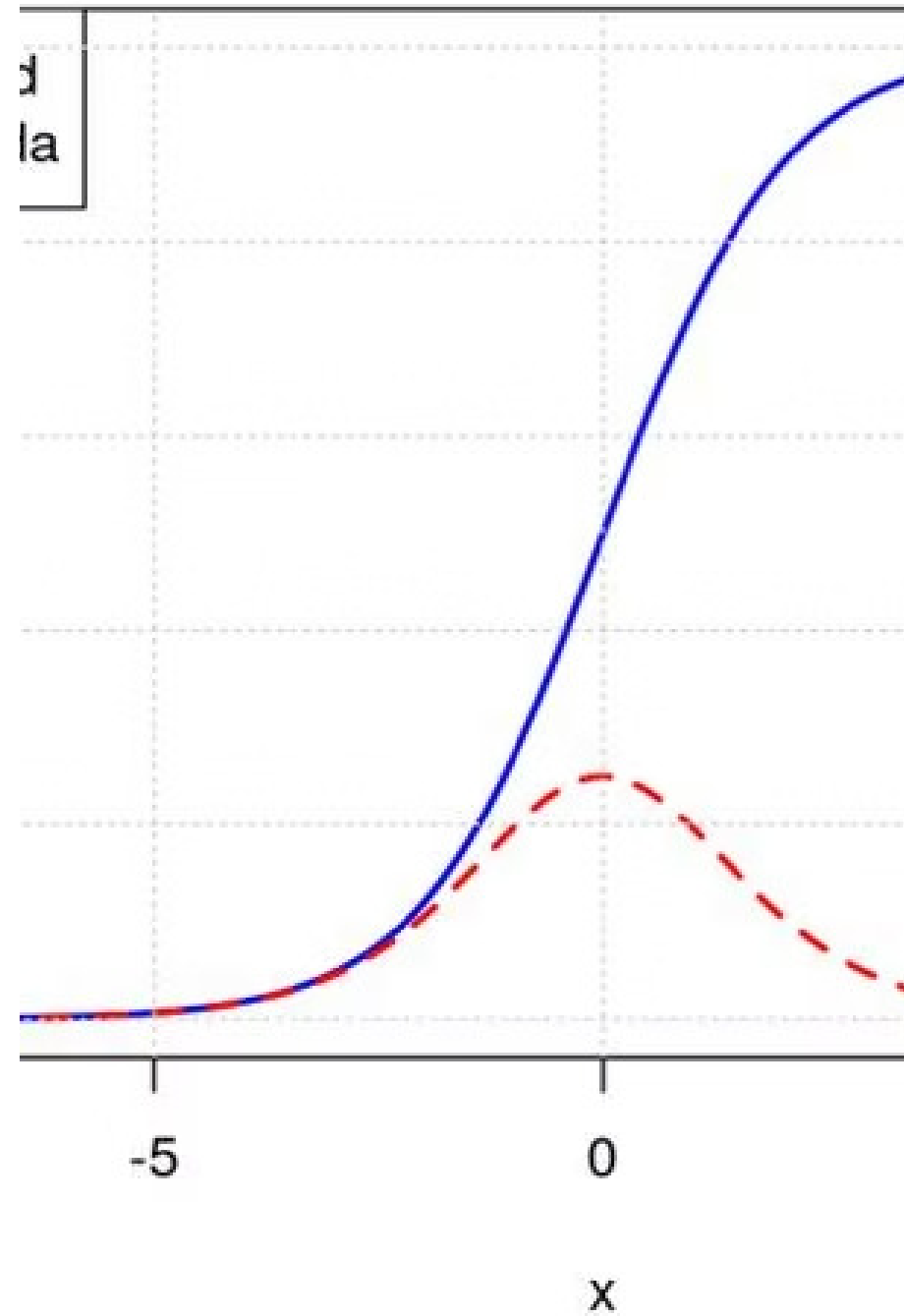
Curva suave en forma de S. Comprime cualquier entrada en el rango (0, 1), expresando resultados como **probabilidades**.

$$f(z) = \frac{1}{1 + e^{-z}}$$

⊗ Sufre **desvanecimiento del gradiente** (vanishing gradient) cuando los valores de entrada son muy altos o muy bajos.

Gráfico: Función Sigmoide y su Derivada

La línea azul muestra la Sigmoide en forma de "S", que pasa por $(0, 0.5)$ y se aproxima a 1 para x grande. La línea roja discontinua es su derivada: una campana simétrica con máximo de **0.25 en $x=0$** , que se aproxima a 0 en los extremos, ilustrando el problema del desvanecimiento del gradiente.



Función Tanh y Función ReLU

Tangente Hiperbólica (Tanh)

Variante de la Sigmoide centrada en cero, oscila entre **-1 y 1**. Acelera la convergencia en modelos multicapa, aunque sigue siendo vulnerable al desvanecimiento del gradiente en sus extremos.

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

ReLU (Unidad Lineal Rectificada)

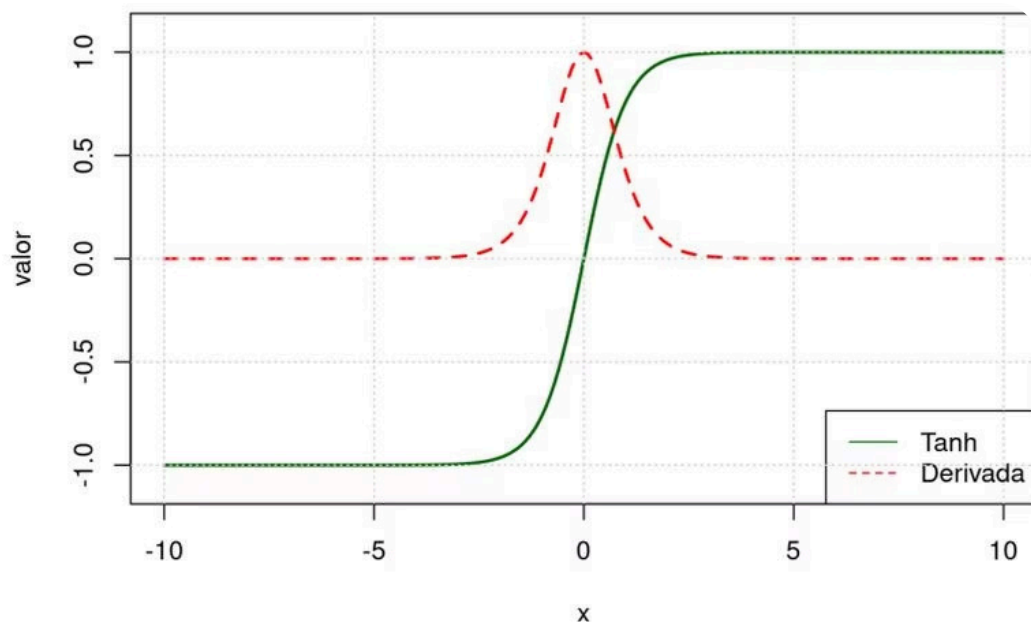
Estándar de la industria. Si el valor es positivo, se transmite sin cambios; si es negativo, se convierte en cero. Extremadamente eficiente y sin desvanecimiento del gradiente para valores positivos.

$$f(z) = \max(0, z)$$

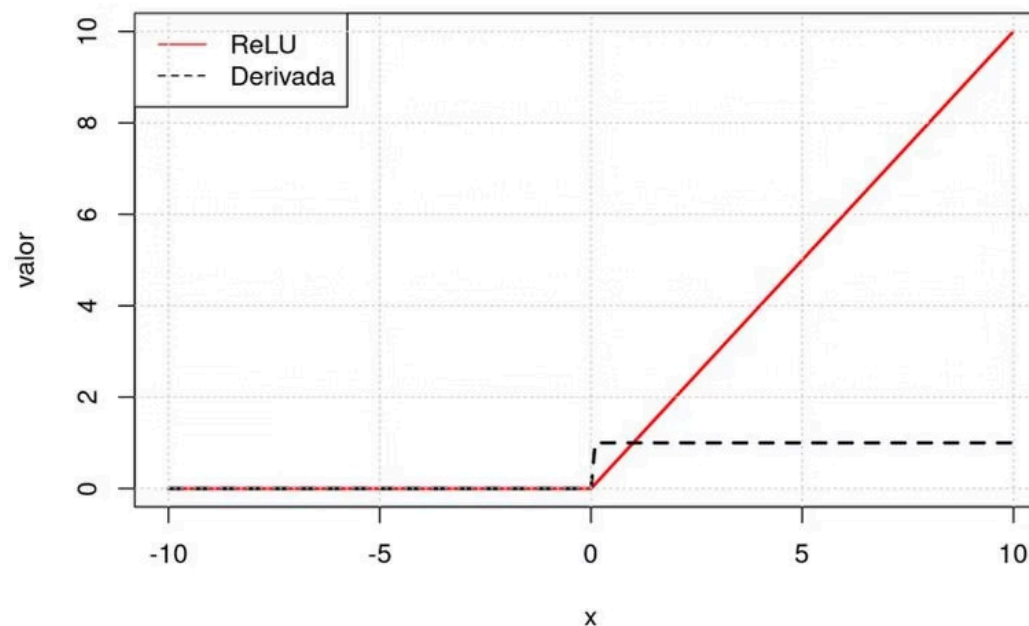
 Riesgo de **neuronas muertas**: si una neurona recibe siempre valores negativos, deja de aprender permanentemente.

Gráficos: Tanh y ReLU

Tanh y su Derivada



ReLU y su Derivada



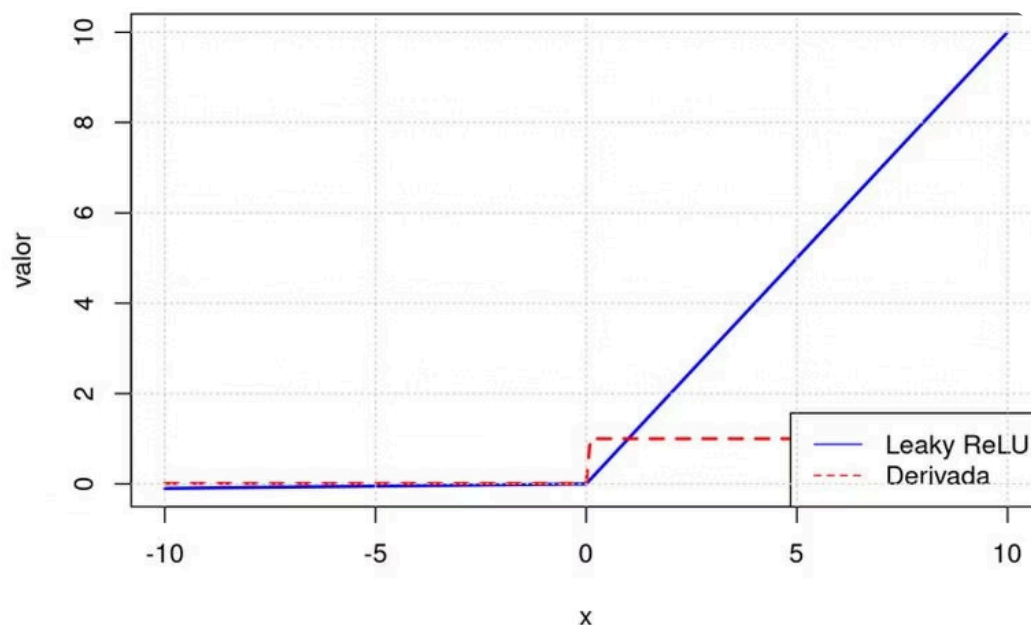
La Tanh (verde) oscila entre -1 y 1 con derivada máxima de 1.0 en $x=0$. La ReLU (roja) es cero para entradas negativas y lineal para positivas, con derivada constante de 1 para $x > 0$, lo que elimina el desvanecimiento del gradiente en esa región.

Leaky ReLU y Softplus: Evoluciones de la Activación Rectificada

Leaky ReLU

Solución al problema de la **neurona muerta**. Permite el paso de una pequeña fracción α (generalmente 0.01) de la señal para valores negativos, garantizando retroalimentación continua.

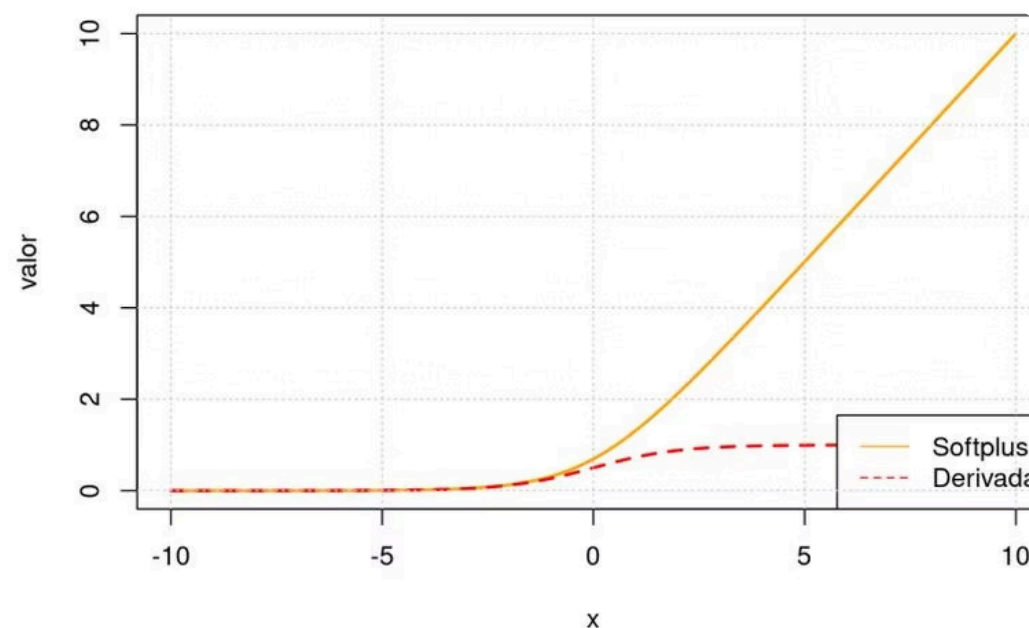
$$f(z) = \max(\alpha z, z)$$



Softplus

Aproximación suave y derivable de ReLU. Su salida siempre es positiva y su derivada es exactamente la **función Sigmoide**, permitiendo un flujo de gradientes muy natural.

$$f(z) = \ln(1 + e^z)$$



Criterios para Elegir una Función de Activación



Capas Ocultas

Usar **ReLU** o **Leaky ReLU** para garantizar velocidad y profundidad en el aprendizaje.



Salida: Clasificación Binaria

Usar **Sigmoide** para obtener una probabilidad entre 0 y 1.



Salida: Clasificación Multiclase

Usar **Softmax**, que asegura que la suma de todas las salidas sea igual a 1.



Modelos de Regresión

Usar **Softplus** cuando se requiere predicción estrictamente positiva con transición suave.

Regularización L1 (Lasso): Simplificación y Selección de Variables

La **Regularización L1 (Lasso)** añade a la función de pérdida la suma de los **valores absolutos** de todos los pesos, generando una penalización constante que empuja los pesos irrelevantes directamente hacia **cero absoluto**.

$$J_{reg} = J + \lambda \sum_{i=1}^n |w_i|$$

Donde **J** es el error original, **λ** controla la intensidad de la penalización y **$|w_i|$** es el valor absoluto de cada peso. A diferencia de L2, la penalización en L1 es constante sin importar qué tan pequeño sea el peso.



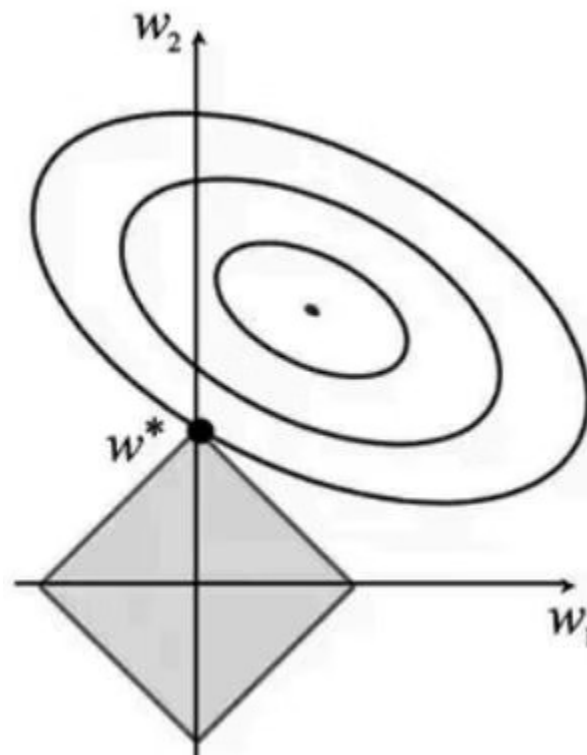
El Efecto de Dispersión (Sparsity) y la Geometría L1 vs L2

Selección Automática de Variables

L1 reduce los pesos de entradas irrelevantes a **exactamente cero**, "apagando" esas conexiones. El resultado es un modelo **disperso** (**sparse**) más robusto e interpretable.

Geometría: El Diamante vs El Círculo

La restricción L1 forma un **diamante** con esquinas sobre los ejes, donde es muy probable que el punto óptimo anule algún peso. La restricción L2 forma un **círculo** curvo, donde es improbable que el contacto ocurra exactamente sobre un eje.



Comparativa: L1, L2 y Elastic Net

L1 – Lasso

Ideal cuando hay muchas variables y solo unas pocas son importantes. **Simplifica la red** y reduce el consumo computacional mediante selección automática de variables.

L2 – Ridge

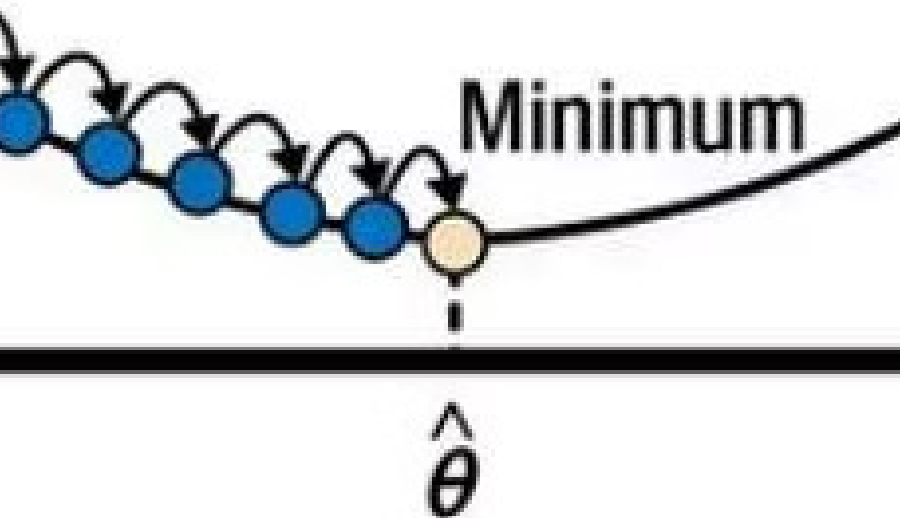
Opción por defecto para **redes profundas**. Previene el sobreajuste de forma suave y estable, manteniendo la capacidad de todas las variables para contribuir al modelo.

Elastic Net

Enfoque **híbrido** que combina ambas penalizaciones: aprovecha la estabilidad de L2 y la capacidad de selección de variables de L1 simultáneamente.

 La regularización no es un accesorio: es el mecanismo que garantiza que el conocimiento extraído sea aplicable a situaciones reales y no una mera memorización de ruidos estadísticos.

tep



La Tasa de Aprendizaje: El Ritmo del Conocimiento

La **Tasa de Aprendizaje (η)** controla el tamaño de cada paso al descender por la función de pérdida hacia el error mínimo. Si imaginamos el aprendizaje como descender una montaña, η es la longitud de cada zancada: un paso mal calculado puede impedir llegar al destino o tardar demasiado.

Impacto de una Tasa Alta vs. Tasa Baja

Tasa Muy Alta

El algoritmo da **saltos gigantes** en la superficie de pérdida. Riesgo de **oscilación y divergencia**: el sistema rebota de un lado a otro superando el punto óptimo, y el error puede aumentar en lugar de disminuir.

Tasa Muy Baja

Ajustes minúsculos que garantizan estabilidad, pero generan dos problemas:

- **Ineficiencia computacional**: requiere enormes cantidades de iteraciones (épocas).
- **Atrapamiento en mínimos locales**: el sistema carece de impulso para escapar de valles poco profundos.

Estrategias de Ajuste Dinámico y Conclusión



Decaimiento de la Tasa (Decay)

El sistema comienza con una tasa alta para explorar rápidamente y la reduce automáticamente para ajustes finales con precisión quirúrgica.



Tasas Adaptativas: Adam / RMSProp

Calculan una tasa distinta para cada peso, permitiendo que las conexiones más importantes aprendan a un ritmo diferente que las secundarias.

Un valor demasiado rígido impide la evolución; uno demasiado elástico destruye el orden necesario para el aprendizaje. El equilibrio de la Tasa de Aprendizaje es lo que permite que una red neuronal **converja** hacia una solución inteligente y generalizable.